

Guía práctica de comandos Lección 08 - Captura de red con tshark

Objetivo y alcance

Usa esta guía como apoyo para los comandos de la actividad. Al finalizar tendrás la captura verificada desde una carpeta compartida de **solo lectura**, el panorama de protocolos y conversaciones, cuatro filtros con su conteo, la sesión HTTP reconstruida, el objeto exportado con su hash, la serie TLS con sus intervalos y la bitácora recuperada en el PC anfitrión.

La captura permanece en `/media/sf_LAB08_ENTREGA/` ; no la copies dentro de Kali. Los archivos de trabajo (resultados de `tshark` , objetos exportados y borrador de la bitácora) se guardan en `~/forense/LAB-08/` . La carpeta compartida `LAB08_SALIDA` , que la guía llama `$RECUPERACION` , solo recibe la bitácora terminada al final de la sesión. Ningún comando de esta guía modifica la captura ni genera tráfico de red.

Todas las órdenes `tshark` usan `-n` : desactiva la resolución de nombres. Sin esa opción, `tshark` intentaría traducir direcciones consultando servicios externos, lo que genera tráfico nuevo y mezcla en el análisis nombres obtenidos hoy con los datos del caso.

Preparación previa: comprobar las herramientas

```
command -v tshark capinfos sha256sum
tshark --version | head -1
```

Kali incluye `tshark` y `capinfos` con Wireshark. Si falta alguno, instálalo antes de aislar la red:

```
sudo apt update
sudo apt install -y tshark
command -v tshark capinfos
```

Parte	Explicación
<code>command -v ...</code>	Muestra la ruta de cada ejecutable. Si falta una ruta, la herramienta no está instalada.
<code>tshark --version</code> <code> head -1</code>	Muestra solo la primera línea, que contiene la versión.
<code>apt install -y</code> <code>tshark</code>	Instala <code>tshark</code> y <code>capinfos</code> . Si aparece la pregunta «Should non-superusers be able to capture packets?», responde No : en esta lección no se captura tráfico, solo se lee un archivo.

No ejecutes `tshark` con `sudo` : leer una captura no requiere privilegios y ejecutar un analizador de protocolos como administrador aumenta el riesgo si el archivo contiene datos malformados.

Variables que usa esta guía

Una variable se escribe `NOMBRE=valor` al definirla y `$NOMBRE` al usarla. **Solo existe en la terminal donde se definió**: si abres otra ventana, los comandos fallarán con rutas vacías.

Variable	Dónde se define	Qué contiene
<code>LAB</code>	Paso 1	Carpeta de trabajo dentro de Kali.
<code>COMPARTIDA</code>	Paso 1	Carpeta compartida con la evidencia; debe rechazar la escritura.
<code>RECUPERACION</code>	Paso 1	Carpeta compartida con escritura donde se recupera la bitácora.
<code>PCAP</code>	Paso 2	Ruta de la captura.
<code>IP_INVESTIGADA</code> , <code>ZONA_CASO</code> , <code>CASO</code>	Paso 2, al cargar <code>RED-CAS002-caso.env</code>	Parámetros confirmados del caso.
<code>STREAM</code>	Paso 5	Número de <code>tcp.stream</code> de la sesión HTTP que identifiques.

Los bloques que guardan archivos comienzan con `cd "$LAB"` . Así, `notas/` y `exportados/` apuntan siempre a la carpeta de trabajo, aunque antes te hayas movido a otra carpeta para revisar un archivo.

Si abres otra terminal

```
LAB=~/forense/LAB-08
COMPARTIDA=/media/sf_LAB08_ENTREGA
RECUPERACION=/media/sf_LAB08_SALIDA
PCAP="$COMPARTIDA/evidencia/RED-CAS002.pcap"
cd "$LAB"
. "$COMPARTIDA/notas/RED-CAS002-caso.env"
echo "caso=$CASO ip=$IP_INVESTIGADA zona=$ZONA_CASO"
```

Si continúas en otra clase

Los PC del laboratorio se restablecen al reiniciarse: lo que quede en el disco del anfitrión, incluida la carpeta `LAB08_SALIDA` , se pierde al apagarlo. Para retomar una bitácora ya iniciada:

- 1. descomprime de nuevo `LAB08_ENTREGA.zip` en Windows y configura las dos carpetas compartidas;
- 2. coloca en la carpeta de `LAB08_SALIDA` la bitácora que guardaste al terminar la clase anterior;
- 3. en Kali, ejecuta el paso 1 completo y luego recupera tu bitácora sobre la plantilla vacía:

```
cp "$RECUPERACION/bitacora-equipo.md" "$LAB/entrega/bitacora-equipo.md"
```

Después ejecuta el paso 2 para cargar los parámetros. Las salidas de `tshark` se regeneran en segundos repitiendo los pasos que necesites.

1. Comprobar las carpetas compartidas y preparar el trabajo

```
LAB=~/forense/LAB-08
COMPARTIDA=/media/sf_LAB08_ENTREGA
RECUPERACION=/media/sf_LAB08_SALIDA

mkdir -p "$LAB"/{exportados/http,notas,entrega}
cd "$LAB"
pwd
if touch "$COMPARTIDA/.prueba-escritura" 2>/dev/null; then
  rm -f "$COMPARTIDA/.prueba-escritura"
  echo 'ERROR: LAB08_ENTREGA admite escritura'
else
  echo 'OK: LAB08_ENTREGA rechaza escritura'
fi | tee notas/carpeta-evidencia.txt
if touch "$RECUPERACION/.prueba-escritura" 2>/dev/null; then
  rm -f "$RECUPERACION/.prueba-escritura"
  echo 'OK: LAB08_SALIDA admite escritura'
else
  echo 'ERROR: LAB08_SALIDA no admite escritura'
fi | tee notas/carpeta-recuperacion.txt
test -r "$COMPARTIDA/evidencia/RED-CAS002.pcap" && echo 'OK: captura accesible' || echo 'ERROR: captura no accesible'
cp -n "$COMPARTIDA/documentos/plantilla-bitacora-leccion-08.md" entrega/bitacora-equipo.md
```

Parte	Explicación
<code>LAB=...</code> , <code>COMPARTIDA=...</code> y <code>RECUPERACION=...</code>	Guardan las tres rutas usadas durante la sesión.
<code>mkdir -p</code>	Crea las carpetas locales que falten sin borrar las existentes.
<code>{exportados/http,notas,entrega}</code>	Separa los objetos exportados, las salidas técnicas y la bitácora.
<code>touch ../.prueba-escritura</code>	Intenta crear un archivo vacío y oculto. En <code>LAB08_ENTREGA</code> debe fallar; en <code>LAB08_SALIDA</code> debe funcionar.
<code>rm -f</code>	Si el archivo de prueba llegó a crearse, lo elimina de inmediato.

Parte	Explicación
<code>if ...; then ... else ... fi</code>	Muestra <code>OK</code> o <code>ERROR</code> según el resultado de la prueba.
<code>test -r</code>	Comprueba que la captura se puede leer.
<code>cp -n</code>	Copia la plantilla sin reemplazar una bitácora que ya exista.

No uses `findmnt` para esta comprobación. Cuando el anfitrión es Windows, Kali suele mostrar ambas carpetas compartidas como `rw`, aunque VirtualBox tenga marcada la opción **Solo lectura**: el sistema invitado no recibe esa marca, pero el anfitrión rechaza cada escritura. Por eso se prueba escribiendo.

No continúes si aparece algún `ERROR`: `LAB08_ENTREGA` admite escritura, la captura no es accesible o `LAB08_SALIDA` no admite escritura.

2. Verificar la captura y leer los parámetros

```
cd "$LAB"
(cd "$COMPARTIDA/evidencia" && sha256sum -c "$COMPARTIDA/notas/RED-CAS002.sha256") | tee
notas/verificacion-sha256.txt
tshark --version | head -1 | tee notas/version-tshark.txt
sed -n '1,20p' "$COMPARTIDA/notas/RED-CAS002-caso.env"
. "$COMPARTIDA/notas/RED-CAS002-caso.env"
PCAP="$COMPARTIDA/evidencia/RED-CAS002.pcap"
echo "caso=$CASO ip=$IP_INVESTIGADA zona=$ZONA_CASO"
less "$COMPARTIDA/notas/RED-CAS002-captura.txt"
```

Parte	Explicación
<code>(cd ... && sha256sum -c ...)</code>	Compara la captura con el hash del manifiesto desde la carpeta de evidencia y regresa a <code>LAB-08</code> .
<code> tee archivo</code>	Muestra el resultado y guarda una copia.
<code>. archivo.env</code>	Carga los parámetros del caso en la terminal actual.
<code>PCAP=...</code>	Guarda la ruta de la captura para no repetirla.
<code>less</code>	Abre la ficha técnica sin modificarla. Se sale con <code>q</code> .

La captura debe indicar `OK` y el `echo` debe mostrar `RED-CAS002`, `10.8.20.45` y `America/Santiago`. Si algo aparece vacío o distinto, detente e informa al docente.

3. Panorama de la captura

```
cd "$LAB"
TZ=UTC capinfos -c -a -e -u "$PCAP" | tee notas/capinfos.txt
tshark -r "$PCAP" -n -q -z io,phs | tee notas/protocolos.txt
tshark -r "$PCAP" -n -q -z conv,ip | tee notas/conversaciones-ip.txt
```

Parte	Explicación
<code>TZ=UTC capinfos -c -a -e -u</code>	Muestra cantidad de paquetes (<code>-c</code>), primer paquete (<code>-a</code>), último paquete (<code>-e</code>) y duración (<code>-u</code>). <code>TZ=UTC</code> fuerza la presentación en UTC; sin ello, <code>capinfos</code> puede mostrar la zona local de Kali.
<code>tshark -r "\$PCAP"</code>	Lee la captura desde el archivo, sin capturar tráfico.
<code>-q</code>	Omite la lista de paquetes y muestra solo la estadística pedida.
<code>-z io,phs</code>	Jerarquía de protocolos: cuántos paquetes contiene cada capa y protocolo.
<code>-z conv,ip</code>	Conversaciones IPv4: cada par de direcciones con sus paquetes, bytes, inicio relativo y duración.

En la tabla de conversaciones, la columna `Total` indica los paquetes y bytes de ambos sentidos. `Relative Start` se mide en segundos **desde el primer paquete de la captura**, no desde la medianoche. Filtra mentalmente las filas donde aparece `10.8.20.45`.

4. Aplicar y contar los filtros

Un filtro de visualización selecciona paquetes ya capturados. Se escribe con `-Y` y no modifica el archivo.

```
cd "$LAB"
F1="dns.flags.response == 1 && ip.dst == $IP_INVESTIGADA"
F2="http.request"
F3="http.response.code >= 400"
F4="tls.handshake.type == 1 && ip.src == $IP_INVESTIGADA"

for n in {1..4}; do
    eval "filtro=\$F$n"
    tshark -r "$PCAP" -n -t ud -Y "$filtro" > "notas/filtro-$n.txt"
    printf 'Filtro %s: %s -> %s paquetes\n' "$n" "$filtro" "$(wc -l < "notas/filtro-$n.txt")"
done | tee notas/filtros-conteo.txt
```

Parte	Explicación
F1	Respuestas DNS (<code>dns.flags.response == 1</code>) dirigidas al equipo investigado.
F2	Solicitudes HTTP de cualquier equipo.
F3	Respuestas HTTP con código de error de cliente o servidor.
F4	Mensajes <code>Client Hello</code> de TLS (<code>tls.handshake.type == 1</code>) enviados por el equipo investigado.
<code>&&</code> dentro del filtro	Exige que se cumplan ambas condiciones.
<code>eval</code> <code>"filtro=\\$F\$n"</code>	Toma el contenido de <code>F1</code> , <code>F2</code> , <code>F3</code> o <code>F4</code> según la vuelta del bucle.
<code>-t ud</code>	Muestra la hora de cada paquete como fecha y hora UTC.
<code>wc -l</code>	Cuenta las líneas del resultado: una por paquete.

Revisa cada resultado con `less notas/filtro-1.txt` y así sucesivamente. Copia a la bitácora el filtro tal como está escrito entre comillas; así otra pareja puede repetirlo.

5. Reconstruir la sesión HTTP

Primero relaciona cada solicitud con su conexión y cada respuesta con su solicitud:

```
cd "$LAB"
tshark -r "$PCAP" -n -Y "http.request" -T fields -E header=y \
    -e frame.number -e frame.time_utc -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
    -e tcp.stream -e http.host -e http.request.uri -e http.user_agent \
    | tee notas/http-solicitudes.txt

tshark -r "$PCAP" -n -Y "http.response" -T fields -E header=y \
    -e frame.number -e frame.time_utc -e tcp.stream -e http.response.code \
    -e http.content_type -e http.content_length -e http.request_in \
    | tee notas/http-respuestas.txt
```

Parte	Explicación
<code>-T fields</code>	Muestra solo los campos pedidos, separados por tabulación.
<code>-E header=y</code>	Agrega una primera línea con el nombre de cada campo.
<code>-e campo</code>	Indica un campo. <code>frame.number</code> es el número de paquete que debes citar.
<code>tcp.stream</code>	Índice que Wireshark asigna a cada conexión TCP de la captura. No viaja en el paquete; lo calcula la herramienta.
<code>http.request_in</code>	En una respuesta, indica el paquete de la solicitud que responde.

Ahora guarda en la variable `STREAM` el `tcp.stream` de las solicitudes HTTP enviadas por el equipo investigado:

```
cd "$LAB"
STREAM=$(tshark -r "$PCAP" -n -Y "http.request && ip.src == $IP_INVESTIGADA" -T fields -e tcp.stream | sort -u)
echo "stream=$STREAM"
```

Parte	Explicación
<code>\$(...)</code>	Ejecuta el comando interior y guarda su resultado en <code>STREAM</code> .
<code>-Y "http.request && ip.src == \$IP_INVESTIGADA"</code>	Selecciona solo las solicitudes HTTP enviadas por el equipo investigado.
<code>-e tcp.stream</code>	Muestra únicamente el número de conexión de cada solicitud.
<code>sort -u</code>	Ordena y elimina repeticiones: si las dos solicitudes usan la misma conexión, queda un solo número.

El `echo` debe mostrar **un solo número**. Compruébalo con la columna `tcp.stream` de `notas/http-solicitudes.txt` y regístralo en la bitácora junto con el filtro que lo produjo. Si aparece vacío, se perdieron las variables: repite el bloque «Si abres otra terminal». Si aparecen dos números, consulta al docente antes de continuar.

Después muestra la conexión completa:

```
cd "$LAB"
tshark -r "$PCAP" -n -t ud -Y "tcp.stream eq $STREAM" | tee notas/stream-http.txt
tshark -r "$PCAP" -n -q -z follow,tcp,ascii,$STREAM | tee notas/stream-http-contenido.txt
less notas/stream-http-contenido.txt
```

La primera orden lista los paquetes de la conexión con su hora UTC; la segunda reúne los bytes transmitidos en ambos sentidos y los presenta como texto, igual que **Follow TCP Stream** en Wireshark. Las líneas con un número solo, como `196` , indican cuántos bytes envió ese extremo en ese tramo.

Los dos paquetes DNS de la sesión están en `notas/filtro-1.txt` : son la respuesta que precede a la conexión y, un paquete antes, la consulta correspondiente.

6. Normalizar una hora

Copia una hora UTC de las salidas y conviértela con la zona del caso:

```
TZ="$ZONA_CASO" date --date='2026-09-14T15:00:00.000000Z' '+%Y-%m-%dT%H:%M:%S.%6N%:Z'
```

Parte	Explicación
<code>TZ="\$ZONA_CASO"</code>	Aplica <code>America/Santiago</code> solo a este comando, sin cambiar la configuración de Kali.
<code>date --date='...Z'</code>	Interpreta la hora indicada. La <code>Z</code> final indica UTC.
<code>%6N</code>	Conserva los seis decimales de segundo que registra la captura.
<code>%:Z</code>	Muestra el desplazamiento respecto de UTC, por ejemplo <code>-03:00</code> .

Conserva en la bitácora la hora UTC y la hora local. Wireshark, en su vista por defecto, muestra la hora en la zona configurada en Kali: no la uses como sustituto de la conversión.

7. Exportar el objeto sin abrirlo

```
cd "$LAB"
tshark -r "$PCAP" -n -q --export-objects http,exportados/http
ls -l exportados/http | tee notas/objetos-exportados.txt
sha256sum exportados/http/* | tee notas/objetos-sha256.txt
```

Parte	Explicación
<code>--export-objects http,exportados/http</code>	Reconstruye los cuerpos de las respuestas HTTP y los guarda en la carpeta indicada, dentro de <code>LAB-08</code> .
<code>ls -l</code>	Muestra nombre y tamaño en bytes de cada objeto exportado.

Parte	Explicación
sha256sum	Calcula el hash de cada objeto sin abrirlo ni interpretarlo.

tshark nombra cada objeto a partir del recurso solicitado. Para saber de qué respuesta proviene cada archivo, compara su tamaño con http.content_length y su nombre con http.request.uri en las salidas del paso 5. No hagas doble clic sobre los archivos, no los abras con un editor asociado y no los ejecutes.

8. Medir la serie TLS

```
cd "$LAB"
tshark -r "$PCAP" -n -Y "tls.handshake.type == 1 && ip.src == $IP_INVESTIGADA" -T fields -E header=y \
-e frame.number -e frame.time_utc -e tcp.stream -e ip.dst \
-e tls.handshake.extensions_server_name -e frame.time_delta_displayed \
| tee notas/tls-client-hello.txt
```

Parte	Explicación
tls.handshake.extensions_server_name	Nombre de servidor (SNI) que el cliente declara en claro al iniciar TLS.
frame.time_delta_displayed	Segundos transcurridos desde el paquete mostrado anterior. Con este filtro, es el intervalo entre un Client Hello y el siguiente. En la primera fila aparece vacío.

La columna Protocol de Wireshark puede mostrar TLSv1.2 en el Client Hello aunque la sesión use TLS 1.3: por compatibilidad, ese mensaje declara una versión antigua y anuncia las versiones reales en una extensión. No interpretes esa etiqueta como un error.

9. Ampliación guiada

Realiza solo una ampliación y únicamente después de completar el producto obligatorio.

Respuesta DNS sin resultado

```
cd "$LAB"
tshark -r "$PCAP" -n -t ud -Y "dns.flags.rcode == 3" | tee notas/dns-nxdomain.txt
```

rcode == 3 corresponde a NXDOMAIN: el nombre consultado no existe en el resolutor.

Fabricante asociado a una MAC

```
tshark -r "$PCAP" -n -Y arp -T fields -e frame.number -e eth.src -e eth.src.oui_resolved -e arp.src.proto_ipv4
```

Los tres primeros bytes de una MAC (OUI) identifican al fabricante registrado de la interfaz. También puedes consultarlo en la herramienta **Wireshark OUI Lookup** (<https://www.wireshark.org/tools/oui-lookup.html>). Una MAC puede cambiarse por software y solo es visible dentro del mismo segmento de red.

Paquetes con observaciones del análisis TCP

```
tshark -r "$PCAP" -n -t ud -Y "tcp.analysis.flags"
```

Comparación con el equipo de control

```
tshark -r "$PCAP" -n -Y "tls.handshake.type == 1" -T fields -E header=y \
-e frame.number -e frame.time_utc -e ip.src -e ip.dst -e
tls.handshake.extensions_server_name
tshark -r "$PCAP" -n -q -z conv,tcp
```

10. Recuperar la entrega

```
cd "$LAB"
cp "$LAB/entrega/bitacora-equipo.md" "$RECUPERACION/"
cmp -s "$LAB/entrega/bitacora-equipo.md" "$RECUPERACION/bitacora-equipo.md" \
    && echo 'OK: bitácora recuperada en LAB08_SALIDA' \
    || echo 'ERROR: las copias de la bitácora no coinciden'
```

Parte	Explicación
cp ... "\$RECUPERACION/"	Copia únicamente la bitácora a la carpeta compartida con Windows.
cmp -s	Compara ambas copias byte a byte. El mensaje OK confirma que coinciden.

No copies a LAB08_SALIDA los objetos exportados ni la captura.

Con el mensaje OK, la bitácora ya está en Windows, pero todavía no está a salvo. Los PC del laboratorio se restablecen al reiniciarse: lo que quede en el disco del anfitrión, incluida la carpeta LAB08_SALIDA, se pierde al apagarlo. Antes de apagar:

1. abre en Windows la carpeta de LAB08_SALIDA y comprueba que contiene bitacora-equipo.md ;
2. entrégala o respáldala por el medio que indique el docente: aula virtual, correo o pendrive;
3. comprueba que la copia entregada se abre correctamente.

No apagues ni reinicies el PC hasta completar esos tres pasos.

Equivalencias en Wireshark

Si el docente modela la sesión en la interfaz gráfica, estas son las mismas operaciones:

Operación	tshark	Wireshark
Desactivar resolución de nombres	-n	View > Name Resolution: desmarcar todas las opciones
Hora UTC	-t ud	View > Time Display Format > UTC Date and Time of Day
Jerarquía de protocolos	-z io, phs	Statistics > Protocol Hierarchy
Conversaciones	-z conv, ip	Statistics > Conversations, pestaña IPv4
Filtro de visualización	-Y "filtro"	Barra de filtro superior
Seguir una conexión	-z follow, tcp, ascii, N	Clic derecho > Follow > TCP Stream
Exportar objetos HTTP	--export-objects http, carpeta	File > Export Objects > HTTP

Qué llevar a la bitácora

Evidencia mínima	Qué registrar
verificacion-sha256.txt y version-tshark.txt	Captura OK y versión usada.
capinfos.txt, protocolos.txt y conversaciones-ip.txt	Datos del panorama; no copies las salidas completas.
filtros-conteo.txt	Los cuatro filtros exactos y sus conteos.
http-solicitudes.txt, http-respuestas.txt y stream-http.txt	Paquetes de las ocho filas de la sesión.
objetos-exportados.txt y objetos-sha256.txt	Nombre, tamaño y hash del objeto.
tls-client-hello.txt	Serie de Client Hello con sus intervalos.
LAB08_SALIDA/bitacora-equipo.md	Copia en Windows que luego se entrega por aula virtual, correo o pendrive antes de apagar el PC.

Si aparece un error

Situación	Qué revisar
No existe <code>/media/sf_LAB08_ENTREGA</code>	Apaga Kali y revisa en VirtualBox el nombre y el montaje automático de la carpeta.
<code>ERROR: LAB08_ENTREGA admite escritura</code>	Detente. Apaga Kali, marca Solo lectura en la carpeta compartida de VirtualBox y repite el paso 1. Que <code>findmnt</code> muestre <code>rw</code> no es, por sí solo, un error.
La captura muestra <code>FAILED</code>	Detente; no vuelvas a calcular ni reemplaces el manifiesto.
Un filtro muestra <code>0</code> paquetes y debería tener resultados	Se perdieron las variables: repite el bloque «Si abres otra terminal». Revisa también comillas y espacios alrededor de <code>==</code> .
<code>tee: notas/...: No such file or directory</code>	Ejecutaste el bloque desde otra carpeta. Ejecuta <code>cd "\$LAB"</code> y repite el bloque. Si ya exportaste objetos fuera de lugar, elimina la carpeta <code>exportados</code> que se creó dentro de <code>notas</code> .
<code>cannot be converted to Unsigned integer</code> o <code>Invalid parameter</code> en el paso 5	La variable <code>STREAM</code> quedó vacía o con texto. Repite el bloque que la define y comprueba que el <code>echo</code> muestre un solo número.
<code>tshark: "... is neither a field nor a protocol name</code>	El nombre del campo está mal escrito. Cópialo desde esta guía.
<code>Running as user "root"</code>	Ejecutaste <code>tshark</code> con <code>sudo</code> . Repite el comando sin <code>sudo</code> .
<code>frame.time_utc</code> no existe en tu versión	Usa <code>-e frame.time_epoch</code> y convierte con <code>date --date=@VALOR</code> . Informa la versión al docente.
<code>--export-objects</code> no crea archivos	Comprueba que la carpeta <code>exportados/http</code> existe y que ejecutaste el comando desde <code>LAB-08</code> .
La hora de Wireshark no coincide con <code>tshark -tud</code>	Wireshark está mostrando la hora local de Kali. Cambia el formato a UTC.